

KAIF — Krinik AI Framework

External memory and discipline for AI agents — in one self-deploying file.

ENGLISH РУССКИЙ

License MIT Version 2.9 Install thin, by machinery Contours 9 Principles 16

General · Installation · Deployed framework · Skills · Working · Updating · Reference

KAIF — Krinik AI Framework — a context-resilient, fundamental strategic-operational methodological framework for AI agents: resilience to context loss and discipline of autonomy.

KAIF is an agent harness — the working environment around the model that lets it carry a long task across tools, steps and sessions. In KAIF the harness is markdown: documents, rituals and guards that the agent reads and maintains.

The project's history lives in the releases and in section 8.1.



Version 2.9 — Distilled KAIF · 2026-10-16

> *KAIF has become a team. A team of AI agents, each taking on the role of an expert professional, builds your project on the git worktree technology. In several VS Code windows at once, like a close-knit crew of specialists, the agents work and discuss the working questions among themselves. You — the owner, the customer — remain the visionary. Your AI agents deliver your order for your project.*
>
> *— Mikalai Kryvusha on KAIF 2.4*

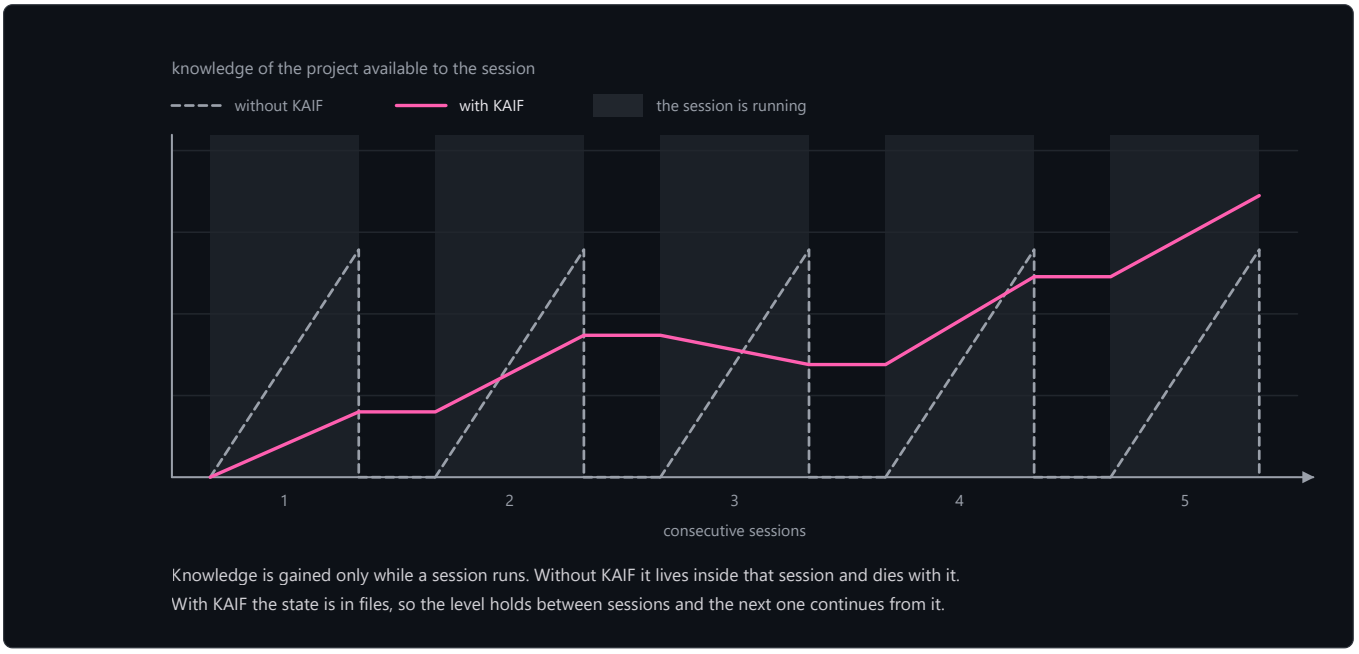
> *A very large-scale version, the one that endows KAIF with the power of an intelligent system that develops itself through a feedback loop. The ouroboros stands for the closing of the loop and for completeness. The eye stands for KAIF now being under the observation of the projects that use it — and those projects being under the observation of KAIF. Every cycle closes — the metaphorical ones and the technical ones alike.*
>
> *— Mikalai Kryvusha on KAIF 2.2*

1. General

1.1. Basic provisions

1. KAIF (Krinik AI Framework) is a methodological framework for AI agents: external memory and discipline for a project, packed into markdown files that the agent reads and maintains.
2. The framework is not code. It is a working process captured as files: key documents, directory conventions, and repeatable slash-skills. It works with any language, any stack, and any domain — programming, science, design, business.
3. The industry has settled on a formula for this layer: Agent = Model + Harness. The models have converged in quality, so the harness around the model now does most of the work. KAIF is an outer harness assembled in markdown, and it moves between agent systems together with the repository.
4. The framework solves two chronic failures of AI agents. Context loss: without external memory, every new session re-discovers the architecture, the decisions, and the half-finished work. Drift: left autonomous, an agent either stalls or oversteps into decisions that are not its to make.

5. The human remains the visionary; the agent executes. Decisions that shape the product (brand, UX, architecture, naming) are made by the owner through interview documents; everything cheap to reverse is decided by the agent autonomously.
6. Nothing raw is trusted. Execution follows the fable loop, everything created carries a test-status marker (`[NOT-TESTED]` / `[TESTED: …]`), and an adversarial judge re-runs the claims before work counts as done.
7. The framework has a full lifecycle: deploy → update → fork → remove. Updates are mechanical and respect every local change (section 6).



1.2. Terms

- **Deployment** — a project into which the framework has been unpacked; recorded in `.kaif/kaif.json` .
- **Owner** — the human whose project it is; the visionary. The owner fills `GOAL.md` , answers interviews, and gives verdicts on taste-class questions.
- **Agent** — the AI system working in the project (Claude or any other); the executor.
- **Skill** — a repeatable ritual the agent performs on command (`/resume` , `/release` , …); the verbs of project work. The full set is given in Table 3.
- **Sphere** — a domain library (programming, science, design, business) that adapts the framework's discipline to the domain's terminology, evidence rules, and fraud table.
- **Canon** — the binding documents of the deployment (Table 1); when canon and improvisation disagree, canon wins.

1.3. Boundaries of application

1. The framework is applied to cognitive projects driven by an AI agent under a human owner. It is not applied as a runtime library: nothing executes inside the target product.
2. Discipline is enforced by documents, rituals, and optional machine guards — not by the model's memory. A deployment where the agent does not read the canon before tasks receives no benefit.
3. The framework does not make the owner's decisions under any breadth of delegation: identity (names, slogans, brand strings) and taste verdicts remain human-only.

2. Installation

2.1. Basic provisions

1. The entry point is one file: `KAIF.md` (15 KB). The agent reads it and executes three bootstrap steps with forced checkpoints; the file fetches the installer machinery from this repository, and the machinery deploys everything mechanically.
2. The machinery unpacks the documents, generates the skills for five agent systems at once, localizes the owner-facing documents, wires the auto-context pointers, validates itself against a sha256 manifest, and self-cleans. The agent's only cognitive work is the short `KAIF_ADAPTATION_TASK.md` : study the project, fill the maps, derive the plan.

3. The agent reads `KAIF.md` of 15 KB. The full core weighs 700 KB, and the thin install takes its reading off the agent's hands.

2.2. Installation procedure

1. Drop `KAIF.md` into the project root.
2. Tell the agent — in your own words. Any of these works, and each one says something different:

   > "*Deploy KAIF from KAIF.md*" — *the plain one. English payload, tracked to the origin.*

   > "*Deploy KAIF from KAIF.md, my working language is Russian*" — *the documents you read and answer in come out in your language; the payload itself stays English.*

   > "*Deploy KAIF from KAIF.md, I have already written GOAL.md*" — *the agent reads your goal and does not ask about it after the install.*

   > "*Deploy KAIF from KAIF.md in anonymous mode*" — *no origin tracking, no author references.*

   The agent needs a working Node.js and network access to this repository.
3. If the agent's harness asks to approve running the fetched installer — that is the download-and-execute pattern being flagged, as it should be; approve it once. The installer verifies every fetched artifact against `kaif-manifest.json` (sha256) before running.
4. That is the whole of your part — from here KAIF installs itself. The install task has the agent wire the reminders (the fresh-context hooks of section 3.4). The install summary names to the agent the KAIF files your project's `.gitignore` hides (the first commit would leave them out), each with the rule that hides it and the cure. If the project folder has no git, the summary tells the agent to run `git init` and to commit your own files, exactly as they are, before the KAIF files.
5. Afterwards the agent asks about one thing only — the goal of your project, if you have not written `GOAL.md` yourself before installing KAIF. `GOAL.md` is what tells your AI agent which project it is working on and what you want to achieve in the end. That question is about how to adapt KAIF to your project.

An installation is tracked to the origin by default — that is what makes version checks, respectful updates and the field-report loop work out of the box. Want a deployment with no tie to the origin at all — tell the agent in step 2: "*Deploy KAIF from KAIF.md in anonymous mode*". Such a deployment carries no origin tracking and no author references (origin-tied skills are skipped, the author's note is stripped mechanically, a final grep-gate refuses to finish while any identity leak remains) and it refuses to update over the network. No update ever converts a deployment between the two modes.

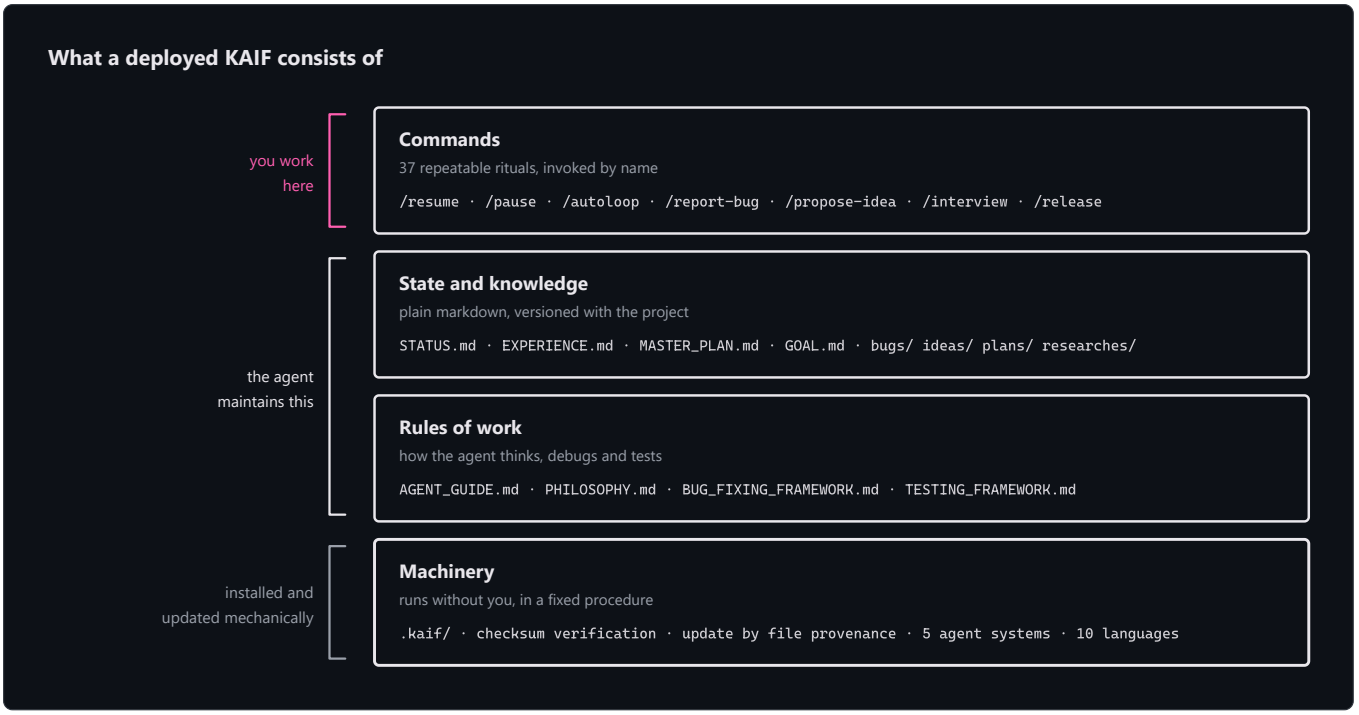
2.3. Offline installation

Every release attaches **`KAIF-FULL.md`** — the classic self-contained core. It unpacks without network access; it is a declared subset of the full delivery — language packs, sphere libraries and the skills' reference templates arrive only with the online path. Prefer the thin path wherever the network exists.

3. The deployed framework

3.1. Basic provisions

A deployment consists of four layers: commands (the skills the human invokes by name), state and knowledge (living documents the agent maintains), rules of work (the canon), and machinery (`.kaif/` — checksums, provenance-driven updates, spheres). Every agent harness carries these layers: instructions, state and memory, verification, guardrails, context management. In KAIF the layers are files, and the harness travels with the repository. The layers are shown in the diagram below.



3.2. The key documents

Fourteen key documents are deployed to the project root. A fifteenth, the owner's voice portrait, is optional and appears where the portrait has been taken.

Table 1 — Key documents of a deployment

Document	What it is for	Who writes / maintains it
`AGENT_GUIDE.md`	The canon: rules, router, checklist, conventions	Machinery deploys; agent adapts; the owner rarely touches it
`PHILOSOPHY.md`	How the agent thinks: KISS + Occam + the principle set	Universal — deployed verbatim
`BUG_FIXING_FRAMEWORK.md`	How the agent debugs: intent gate, fix→build→test, twin check	Universal — deployed verbatim
`TESTING_FRAMEWORK.md`	How the agent tests everything it creates: 7 principles + the `[NOT-TESTED]`/`[TESTED]` contract	Universal — deployed verbatim
`REQUIREMENTS_FRAMEWORK.md`	How the agent writes and checks requirements: goal vector + acceptance criteria first, ten quality criteria, the stop-word dictionary	Universal — deployed verbatim
`GOAL.md`	The vision: what the owner wants in the end	**The owner** — the one document that is theirs to fill
`STATUS.md`	The living summary of now (~200-line soft target)	Agent, after every significant task
`PROJECT_HISTORY.md`	The append-only chronicle of closed sessions, phases, releases	Agent moves entries at `/end-chat-soft`
`EXPERIENCE.md`	The grep-friendly log of paid-for lessons	Agent grows it (`/experience`)
`MASTER_PLAN.md`	The phased roadmap from the current state to `GOAL.md`	Agent derives it (`/revision`)

`PROJECT_STRUCTURE_EXTERNAL_MAP.md`	The external map: directories, files, links	Agent maintains
`PROJECT_ARCHITECTURE_INTERNAL_MAP.md`	The internal map: abstractions and their interactions	Agent maintains
`KAIF_FRAMEWORK.md`	The deployment record: which KAIF is deployed in the project and how	Agent writes after injection
`KAIF_REFERENCE.md` (at `.kaif/`)	The complete framework reference; `/help-kaif` cites its sections	Deployed verbatim
`AUTHOR_STYLOMETRY.md` — **optional**	The portrait of the owner's written voice: registers, rules, an anti-portrait of AI markers, a self-check list. Text the owner signs is held to it	Taken by `/owner-voice` from the owner's own texts; the owner accepts it

3.3. The knowledge directories

Seven knowledge directories are deployed, each with its own README. Closed items in `bugs/`, `ideas/`, `plans/`, and `homeworks/` receive the `DONE` tag in the filename; the living references are never tagged.

Table 2 — Knowledge directories

Directory	What accumulates there
`plans/`	The agent's operational plans and epic ladders
`ideas/`	Feature proposals — mostly the owner's; implemented only after approval
`bugs/`	One document per defect, with forensics
`researches/`	Recon documents and answers to the big hard questions
`interviews/`	Owner-level decisions; the owner answers right in the document
`homeworks/`	Tasks only a human with a body can do, including taste-class artifacts
`reports/`	The agent's reports on cognitively heavy work; mandatory KAIF update/install field reports (`KAIF_UPDATES/`) and strong-model audit reports (`KAIF_AUDIT/`)

3.4. The machinery

1. `.kaif/` holds the deployment marker (`kaif.json`), the machinery (`kaif-core.mjs`, backing the `npm run kaif:*` handles), the sphere libraries, and shipped templates (for example, the owner-voice portrait skeleton `_owner-voice-template.md`).
2. Every deployed file is classified by provenance: template shas record what the template looked like, disk shas record what the file looks like now. The update machinery (section 6) merges by this classification, module by module.
3. The update closes with `update-verify`: per-system skill copies are re-synced from the canonical `.claude/skills/`, placeholders are re-scanned, the deploy marker self-heals, and a receipt is written to `.kaif/last-update.json`.
4. `.kaif/hooks/` holds the **refresh-hooks** module for harnesses with lifecycle hooks: an order to re-read the canon after a context compaction, a timer on the age of the refresh marker, a soft `STATUS.md` guard once per session, and an order to run `/resume` when the owner's message opens with the word `resume`. The install and update tasks wire the module with the `wire-hooks` command; the owner opts out with the key `"hooks": "off"` in `.kaif/kaif.json`, and a deployment without hooks does not redden any gate. The hook `pretool-owner-word.mjs` of version 2.8 stopped the agent's work on every message the owner sent mid-turn; version 2.9 removes it from the delivery: the update deletes its untouched copy and its entry in `.claude/settings.json`.

4. The skills

4.1. Basic provisions

1. A skill is a repeatable ritual invoked by name (`/resume`) or by a natural-language trigger in the owner's language («сделай релиз», «haz un release», …). Trigger aliases in ten languages are appended at deploy time; the skill bodies stay English.
2. Skills are generated for five agent systems at once — Claude Code, Codex, Grok Build, Cline, Zoo Code — plus the universal `AGENTS.md` ; the canonical copies live in `.claude/skills/` .
3. Thirty-seven skills are deployed.

Table 3 — The skills

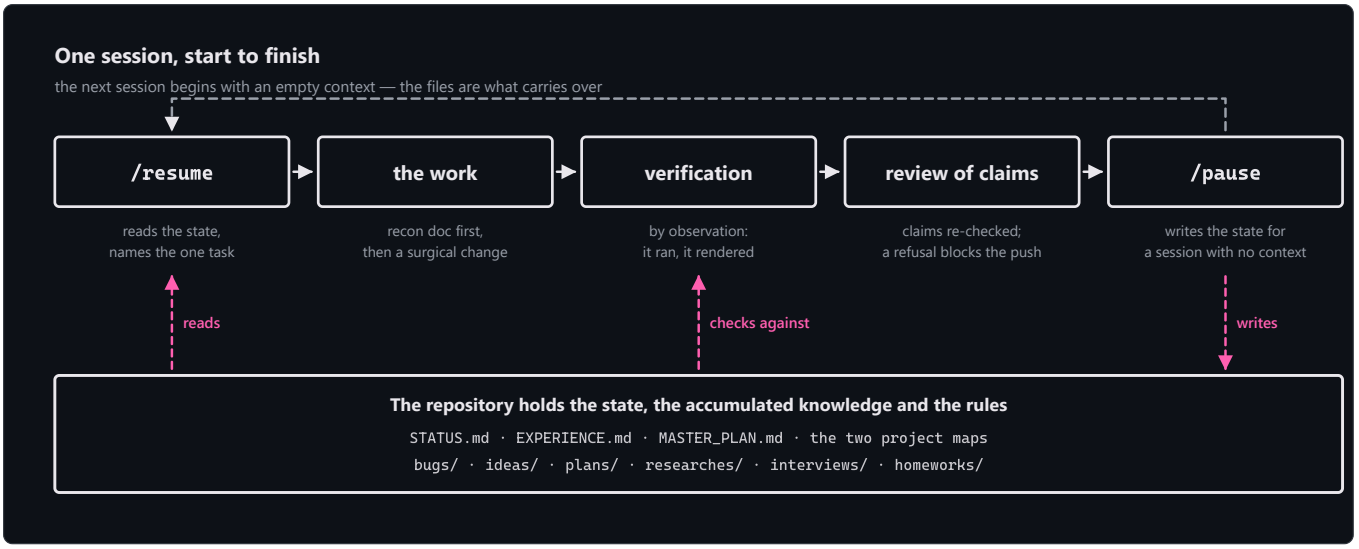
Skill	Purpose
`/resume`	Start a session: read the canon docs, pick the one main thing and announce it. The owner's message under the word `resume` sets the scope of the work: a question gets an answer and no work beyond it, a task gets the task, a bare `resume` gets the chosen main thing.
`/kaif-go`	The slash-command form of saying "carry on" (short alias `/go`): continue the work in the current chat. It is never a blanket yes to vision forks, the write-gate or an `AUTH:` line.
`/pause`	Soft-park the chat: reach a logical stopping point, keep the tree green, continue later in the current chat — no pushes, no ceremony.
`/end-chat-soft`	Softly close the chat: finish the current work to a natural cut, then unhurriedly update `STATUS.md`, rebuild artifacts, commit AND push, hand over to the next chat.
`/end-chat-force`	Urgently close the chat right now: capture only what must not be lost (status + handover), commit AND push; the skipped ceremonies become an explicit debt line.
`/autoloop`	A long autonomous series over the backlog; every item ends with a mandatory judge pass.
`/dayloop`	Daytime autonomous work while the owner is busy — with brief progress pings in chat.
`/nightloop`	Autonomous work until morning; the morning report leads with outcomes.
`/guarded-loop`	An autonomous loop under a watchdog: external wake-ups every N minutes, a heartbeat file proving real progress, a restart policy with an escalation cap.
`/refresh-context`	Re-read the master plan, the maps and the open backlog mid-marathon — rebuild the big picture.
`/check-backlog`	Audit `bugs/` + `ideas/` + `plans/`: list what is open, tag the finished `DONE`.
`/experience`	Capture a lesson into `EXPERIENCE.md` — or recall lessons by tags before a task.
`/report-bug`	File a defect document in `bugs/` by the canon — one file per bug.
`/bug-research`	Deep investigation without code edits — mandatory after 3 failed blind fixes.
`/propose-idea`	Propose a feature as an `ideas/` document — implemented only after the owner's approval.
`/interview`	Ask the owner the fateful A/B/C/D questions — vision decisions are never guessed.
`/owner-voice`	A stylometric portrait of the owner's written voice, taken from their own texts into the optional `AUTHOR_STYLOMETRY.md`; AI text in the owner's artifacts is then written or re-voiced to sound like the owner, under machine-checkable invariants.
`/owner-reviews`	The optional review contour: interviews and outbound drafts rendered as local HTML pages, decisions recorded with author and time, sends mechanically gated by approval — fail-closed.
`/team-deployment`	Design and deploy a TEAM of AI agents: analyze the work profile, suggest an evidence-informed composition the owner approves, then materialize it as isolated worktrees under a generated Team Constitution and a shared status board. Ships role contracts, two archetypes and the board-tool contract; the project's agent builds the tools.

`/fix-vision`	Capture the owner's vision-level chat messages into the docs before they evaporate.
`/what-next`	Rank the next steps by value toward the vision when the owner asks "what now?".
`/plan-task`	Plan an ordinary task/bug/idea into ONE operational plan; heavy tasks are handed to `/plan-epic`.
`/plan-epic`	Plan a heavy epic by the full ladder: industry web-recon + local recon → research doc → meta-plan with phases → operational plan of the NEXT phase only.
`/revision`	Re-derive `MASTER_PLAN.md` from `GOAL.md` and the current state.
`/derive-styleguide`	Derive the owner's style guide from THEIR OWN sample — approved once, then machine-lintable rules guard it.
`/code-revision`	A periodic reading revision of the codebase by the strongest model: parallel reviewers armed with the project's own paid-for failure classes; every finding needs a verbatim quote and survives an adversarial skeptic — or dies. The run leaves audit reports in `reports/KAIF_AUDIT/`, grouped by finding family, each finding a contract a weaker model can execute.
`/fable-method`	The execution loop: classify → define done → evidence → act → verify → report. *(vendored from fable-method, MIT)*
`/fable-loop`	Orchestrated run: parallel evidence, surgical execution, adversarial verifiers.
`/fable-judge`	Adversarial verification of any "done" claim: VERIFIED / CAVEATS / REFUTED.
`/fable-domain`	Generate a trusted domain-workflow bundle (adapter + trap + smoke eval).
`/help-kaif`	Explain KAIF to the owner in chat — a structured user manual.
`/release`	Publish a release (with the owner's confirmation and a mandatory judge pass; never autonomously).
`/kaif-version`	Report the deployed KAIF version and check origin for a newer release.
`/kaif-update`	Mechanical respectful update from origin — content snapshots protect local customizations.
`/kaif-fork`	Snapshot the evolved KAIF into the owner's own repository and track their own line.
`/kaif-switch-origin`	Switch tracking from a fork back to the official origin.
`/kaif-remove`	Respectful removal — asks partial (knowledge artifacts stay) vs full.

5. Working on a project

5.1. The session cycle

A session begins with `/resume` (the agent reads the canon and picks the one main thing), proceeds through the work with verification, and ends with `/pause` (soft-park) or `/end-chat-soft` (full closure with a handoff). The state carries over in the files, not in the chat: the next session starts from an empty context and is productive immediately.



5.2. Roles and interviews

1. The owner is the visionary: fills `GOAL.md`, drops ideas into `ideas/`, answers interviews, gives taste verdicts. The agent is the executor: everything else.
2. Everything the agent wants FROM the owner (a fork, a review, an approval, an answer) lives only in `interviews/`; questions are closed A/B/C/D with the recommendation first, and the owner answers right in the document. An answer given on a rendered page, in the document, or in chat carries equal force and is recorded with author and time.
3. The owner's written voice is reproducible: `/owner-voice` takes a stylometric portrait from the owner's own texts into `AUTHOR_STYLOMETRY.md` (an optional canon file in the project root), and AI text in the owner's artifacts is then held to it.
4. A question to the owner is short: one fork is one question, and the subject of the decision is quoted inside it. The question page behaves as the owner expects: a second press on the chosen option clears it, a paragraph wrapped over several lines is shown as one paragraph, and the owner's answer takes line breaks.

5.3. Autonomy loops

If the owner is away, the agent grinds the backlog autonomously: `/dayloop` (with progress pings), `/nightloop` (morning report), `/autoloop` (a long series), or `/guarded-loop` (under an external watchdog with a heartbeat file — a hung chat cannot silently kill the run). Every item ends with a mandatory judge pass; an owner's drive-by note is filed to the backlog, not a task switch.

5.4. Execution discipline

Any non-trivial task runs the fable loop: classify the ask → define done → gather evidence → decide → act surgically → verify by observation → report outcome-first, with forced artifacts at decision points. When work is declared complete (the agent's own or anyone else's), `/fable-judge` re-runs the claims adversarially before the work counts as done. With git the agent keeps one rule — one command, one action: `git add`, the commit and the push are separate calls; the auto-mode permission classifier refuses a chain of commands joined with `&&`.

5.5. Guardrails and provenance

1. Observation beats conjecture: claims trace to sources, and a gap in the canon opens exactly three doors — find the answer in an existing source of truth, ask the owner, invent something plausible. The third door is shut.
2. AI-written text in the owner's canon artifacts carries provenance marks `[AI]…[/AI]`; only the owner's word removes a mark. Optional machine tools (the provenance gate, the canon linter) ship as modules and are wired on request.
3. A guard of a text rule runs at ~10 false hits per real one; exceptions are explicit, with the reason on the line. A false `[TESTED]` marker is fraud for the judge.

5.6. Fresh context

1. Rules read once at the start of a session melt as the context fills and is compacted: a long session holds a retelling of the canon. So the agent re-reads the canon core on four triggers — an hour of live work, the start of a heavy task, a compaction or a long pause, and the ritual points (`/resume`, `/refresh-context`, every iteration of the loops).

2. A refresh is a verifiable act. Its witness has two halves and both are required: the marker `.kaif/refresh-marker.json` (the moment, what was re-read, which trigger) and an acceptance quote in the chat — one concrete line out of what was re-read, relevant to the task in hand. A marker without a quote is fraud of the same class as a false `[TESTED]`.
3. The agent learns its own machine from its own probes: the environment dossier in `AGENT_GUIDE.md` records what the shells, the toolchain and the encodings actually are, keeps the probe command next to every fact, and declares facts older than four weeks to be hypotheses again.
4. The fresh-context hooks find the project root when the agent has gone into a subfolder too: a hook's reminder concerns the project the agent works in.

6. Updating, forking, removing

6.1. Basic provisions

1. Updates are mechanical and respectful: `npm run kaif:update` (or `node .kaif/kaif-core.mjs update`) fetches the latest machinery and classifies every framework file by provenance. A file never touched locally refreshes wholesale; a file adapted or localized goes through the module-by-module merge — local modules stay local, untouched upstream modules take the new template text silently, and a module lands in the short `KAIF_UPDATE_TASK.md` (with a ready diff) only where upstream actually changed under local edits.
2. The owner's content — `GOAL.md`, `STATUS.md`, the knowledge directories — is never in scope.
3. The update closes with the same guarantees as a fresh install (section 3.4) and leaves a receipt in `.kaif/last-update.json`.
4. The update task `KAIF_UPDATE_TASK.md` lists the places of the project that still refer to KAIF skills and hooks that were removed or moved, and the update brings the context pointers of the other agent systems to this release's text: `AGENTS.md`, `CLAUDE.md`, the Cline and Zoo Code rules. In an update across several versions the project's own edits are marked with the line "with local edits", and `update-verify` names them as the project's edits.
5. The rule changes of a release are accepted with the version: the update task does not put them to the owner as questions and names them in the field report.

6.2. The handles

The mechanical handles installed into `package.json` are given in Table 4. Forking, switching origin, and removal are driven by their skills — ask the agent.

Table 4 — The `npm run kaif:*` handles

Command	What it does
`npm run kaif:version`	Show the deployed KAIF version (from `.kaif/kaif.json`).
`npm run kaif:check`	Validate the deployment against its manifest — works even after self-clean.
`npm run kaif:update`	Mechanical respectful update from origin (section 6.1).

6.3. Updating old deployments

Drop the fresh thin `KAIF.md` on top of a pre-1.5 deployment and ask the agent for an update. The installer detects the existing deployment and adopts everything it finds as local; the owner's content survives such an update byte-for-byte.

7. Spheres, agent systems, languages

7.1. Spheres

A sphere ships to `.kaif/spheres/` with the domain's terminology and its execution discipline: the binding minimum evidence set, the authority order, what "verified by observation" means there, the fraud table the judge hunts by, and the domain's craft recipes. Prebuilt spheres: programming · science · design · business; a new sphere is authored at deploy time from the shipped template.

7.2. Agent systems

Skills are generated for five systems at once: Claude Code (`.claude/skills/`, canonical) · Codex (`.agents/skills/`) · Grok Build (`.grok/skills/`) · Cline (`.cline/skills/`) · Zoo Code (`.roo/commands/`) — plus the universal `AGENTS.md`;

Cursor/Copilot/Windsurf ride the fallback. The refresh-hooks module carries sample hook configurations only for the harnesses whose hook contract was read off live vendor documentation: Codex, Cursor, Copilot, Antigravity; Grok Build matches the Claude Code contract and needs no sample of its own. Where a vendor has no hook mechanism, the module says so in one line instead of guessing a path.

7.3. Languages

The framework's sources are English. On deploy the machinery localizes the owner-facing documents (`GOAL.md` , `KAIF_FRAMEWORK.md` , the directory READMEs) from prebuilt packs — ten languages: en, ru, zh-Hans, es, hi, ar, pt, fr, de, ja — and appends trigger aliases in the owner's language to every skill. Agent-internal documents stay English by design. Other languages degrade honestly: English plus a translation item in the adaptation task.

Since 2.3 two packs are maintained: en and ru. The other eight are frozen at their full 2.2 state — they stay in the delivery, deploy exactly as before and are pinned byte-exact by a guard, but they receive no updates. The reason is practical: nine parallel packs are heavy to maintain and do not yet pay for themselves. A frozen pack is revived on community demand — open an issue.

8. Reference

8.1. Milestones

The history in one table; each codename is a discipline the framework learned. Full notes (from 1.1 on) live in the releases.

Table 5 — Versions

Version	Codename	Date	The discipline learned
v1.0	—	2026-06-30	The distillation: the working method extracted into one self-extracting core; the repository wrapped by its own framework from day one.
v1.1	Structured KAIF	2026-07-01	`x.y` versioning, the key-document set (vision, plan, two maps), the knowledge directories.
v1.2	Anonymous KAIF	2026-07-03	The mechanical unpacker, the anonymous install mode, skill translation for non-Claude systems.
v1.3	Slim KAIF	2026-07-06	A one-file lightweight variant (retired in 1.5 in favor of the thin core + offline `KAIF-FULL.md`).
v1.4	Savvied KAIF	2026-07-08	`EXPERIENCE.md` — the grep-friendly journal of lessons; lazy context loading; optional hook enforcement.
v1.5	Tested KAIF	2026-07-17	The thin install, five agent systems and ten languages at once, mechanical respectful updates, `TESTING_FRAMEWORK.md`, the vendored fable loop; field-certified on a 12 B local model.
v1.6	Homeostatic KAIF	2026-07-24	Guardrails for weak models: observation over conjecture, the three doors, the judge before every push, provenance marks `[AI]…[/AI]`.
v2.0	Excellent KAIF	2026-07-28	Updates by machinery, not by mind: the module map, template-vs-disk shas, update receipts, the `KAIF_REFERENCE.md` reference, the permanent sandbox polygon.
v2.1	Strong KAIF	2026-07-31	The owner contour: the place-of-questions rule with `/owner-reviews`, the owner's voice portrait `/owner-voice`, craft prostheses for weak sessions (`/code-revision`, craft slots, `/guarded-loop`), the planning ladder, the `PROJECT_HISTORY.md` chronicle.
v2.2	Yolden KAIF	2026-08-08	The loop closes: the interactive contour turns a question to the owner into a working channel, the field-to-origin signal path gained five prescribed steps, `REQUIREMENTS_FRAMEWORK.md` joins as the 14th key document, re-reading the canon becomes a verifiable act with a witness marker and the optional `refresh-hooks` module, and `/kaif-go` is the slash-command form of saying "carry on": a simple way to continue the work in the current chat.

v2.3	Subjected KAIF	2026-08-21	The version that spent its cycle under observation: thirteen issues from the agents of live projects became the scope, so this one is made of what its users found. The canon speaks in commands, so an obligation carries a command, a numbered step or a checkbox, and `TESTING_FRAMEWORK.md` is rebuilt around a chain of testing activities; a recorded lesson has to answer whether the trap can be mechanized away instead of remembered; `update` writes a journal before its first mutation, and `resume` finishes what a killed run started; a failed network call on Windows reports one error instead of two; eight language packs are frozen with a declared version, state and reason, while `en` and `ru` stay maintained.
v2.4	Teamed Up KAIF	2026-08-28	The harness takes a team: the optional `/team-deployment` skill designs and deploys a team of AI agents — roles, isolated worktree workplaces, a generated team Constitution, a shared status dashboard. Closing a chat splits into `/end-chat-soft` and `/end-chat-force`, and a named end time starts the soft close instead of cutting the work short. Before work the agent speaks the creed and the prayer of principles. The interactive contour calls with its named neural voice and renders choices as radio buttons. Four field fixes from the 2.3 reports ship alongside.
v2.5	Experienced KAIF	2026-09-04	Rules move to the point of decision: the KAIF-defect carve-out stands inside the authorization gate, an engineering fork is closed by the fourth door and a forced `FORK:` line, every guard declares the threat it was proven against, the guarded loop writes its boundary. Delivery is counted by one owner metric in every session close, defects get a severity ladder, the re-read core gets line budgets. A KAIF ticket is delivered by a machinery command. The team skill adopts a live team, gives the board four states and ships CI with the team. The update keeps an anchored block whole, prints its verdicts with numbers and holds a rehearsal binding, writes the stale-claims item unconditionally and scans the project's scripts. An acceptance criterion can be written in the owner's language as a four-line scenario — Situation · Action · Result · Check — with an optional linter that guards its form.
v2.6	Mindful KAIF	2026-09-06	The agent works on the owner's terms: a question to the owner is priority one and its showing is a recorded fact, every question and option is a scenario of what the owner will see in the customer's language, the delivery vector is derived from a registry of systems and never asked, the update rehearsal reaches the bootstrap route, the interactive contour ships as a spec plus a generator of three faces, a fresh remark is recorded and not ranked, "done" about production comes after the owner's real world, and the delivery names no donor projects.
v2.7	Audited KAIF	2026-09-18	The agent's records become checkable: the owner's word is a quote and the agent's word is signed, a question to the owner is asked after a search through his prior answers, "test" means a functional run and every executed run leaves a report, the owner's text is written by his voice portrait, the owner's page keeps his answer, a falsehood is corrected where it stands, a lesson repeated twice goes red, and the size budget of the documents the agent re-reads at the start of a session counts the lines the project wrote.
v2.8	Noble KAIF	2026-09-26	KAIF becomes lighter and hears the owner: the agent answers the owner's word while it works and keeps working, the owner's answered decisions come before the plan, the owner's question page saves answers one at a time, the update loses nothing silently, the scanners see the project as git sees it, the canon keeps every rule and hands the stories behind its rules to the reference `KAIF_REFERENCE.md`, and a tester's bug report gets a required shape, with "not reproduced" written only after a hunt of at least three variants.
v2.9	Distilled KAIF	2026-10-16	A new version's delivery is wired and accepted at once: the install and update tasks wire the reminders themselves, the rule changes of a release are accepted without an interview; the hook that stopped the agent on the owner's message mid-turn is removed. The update names the places of the project that refer to removed skills and hooks, brings the pointers of the other agent systems up to date and, in an update across several versions, marks the project's own edits. The owner's question page behaves as he expects; a question to the owner is short; to a question under the word `resume` the agent answers and starts no work

			beyond it; the attribution linter keeps the owner's private files out of a shared commit; for git one command does one action.

8.2. Repository layout

```
KAIF.md                                ⭐ the THIN entry point (bootstrap + embedded loader), generated
KAIF_REFERENCE.md                      the complete framework reference (generated from
framework/KAIF_REFERENCE.md)
README.md                              this manual (EN+RU)
README.pdf                             its rendered copy
LICENSE                                MIT
KAIF.jpg                               the logo
framework/                             the canonical universal templates (the payload)
  _intro.md                            the narrative of the full core
  installer/                           KAIF-CORE.mjs (the machinery) · KAIF-LOADER.mjs · the thin core's
narrative
  skills/                              37 skill templates (one directory per skill)
  spheres/                             sphere libraries: programming · science · design · business · _template ·
_index
  adapters/                            10 agent-system adapters (five skill-target systems + fallback and
archived ones)
  templates/_owner-voice-template.md   the owner-voice portrait skeleton (ships to .kaif/)
  templates/languages/                 9 language packs (owner-facing docs + skill trigger aliases; English is
the source)
  tools/                               10 optional tool modules: the provenance gate and nine linters (canon,
requirements,
                                       guards, scenarios, ranking, attribution, run reports, voice, experience) +
the contour generator contour/
  hooks/                               the refresh-hooks module (4 scripts + sample configs → .kaif/hooks/)
  readmes/                             7 directory READMEs
  AGENT_GUIDE.md … KAIF_REFERENCE.md   the fourteen key-document templates
dist/                                  generated distribution (never hand-edited)
  KAIF.md                              the thin entry point
  KAIF-CORE.mjs                        the installer machinery
  KAIF-CORE-BUNDLE.md                  the payload bundle (file blocks)
  kaif-manifest.json                   sha256 manifest
  KAIF-FULL.md                         the offline self-contained core
  kaif-module-map.json                 the module map (headings → modules)
assets/                                generated README diagrams (3 × light/dark × EN/RU)
tools/                                 build-framework.mjs · check-framework.mjs · sandbox-suite.mjs (the
polygon)
                                       · module-map-lib.mjs · build-diagrams.mjs · readme-pdf.mjs · commit.mjs ·
kaif.mjs
AGENT_GUIDE.md · STATUS.md · …         the dogfooding wrapper: the framework applied to itself
plans/ ideas/ bugs/ researches/        the wrapper's knowledge directories
interviews/ homeworks/ reports/        (each with its own README)
```

8.3. This repository is fractal

This repository is the framework and is wrapped by the framework — it uses itself. The root holds a real `AGENT_GUIDE.md`, `STATUS.md`, `.claude/skills/`, and knowledge directories describing the development of the framework itself. A deployment into another project starts from `KAIF.md` only — never from this repository's wrapper files. Everything a deployment needs is fetched from `dist/`, which is generated from `framework/` by `node tools/build-framework.mjs`; generated artifacts are never hand-edited.

8.4. Limitations of the current version

1. Localization covers the owner-facing documents and skill trigger aliases (ten languages); the canon and skill bodies are English by design.
2. Native skills are generated for five agent systems; other harnesses (Cursor, Copilot, Windsurf) ride the universal `AGENTS.md` fallback without native skill files.
3. The sandbox polygon (28 suites) verifies the deploy/update machinery; the methodology itself is verified by field reports, not by the polygon.
4. Discipline is enforced by documents and rituals; without the optional tool modules and hooks there is no runtime enforcement — an agent that skips `/resume` works without the canon.
5. The delivery holds 14 documents + 7 READMEs + 37 skills + 1 unpacker = 59 embedded files; 184 bundle blocks; 836 modules.

License

MIT — © 2026 **Mikalai Kryvusha** aka **KOT KRINIK** · Николай Кривуша aka Кот Криник. The execution-discipline skills (`fable-*`) are vendored from fable-method © Sahir619, MIT.

Use it, copy it, modify it, ship it — including on the framework's own project. Thank you, and pleasant work!

КАИФ — Криник АИ Фреймворк

Внешняя память и дисциплина для ИИ-агентов — в одном самораскрывающемся файле.

ENGLISH РУССКИЙ



Общие сведения · Установка · Устройство · Навыки · Работа · Обновление · Справка

КАИФ — Криник АИ Фреймворк — контекстоустойчивый фундаментальный стратегическо-операционный методологический фреймворк для ИИ-агентов: устойчивость к потере контекста и дисциплина автономности.

KAIF — это агентный харнесс (обвязка): рабочее окружение вокруг модели, которое даёт ей вести долгую задачу через инструменты, шаги и сессии. В KAIF харнесс — это markdown: документы, ритуалы и стражи, которые агент читает и ведёт.

История проекта живёт в релизах и в разделе 8.1.



Версия 2.9 — Distilled KAIF · 16.10.2026

> *KAIF стал командным. Команда ИИ-агентов, каждый из которых принимает на себя роль профессионала-эксперта, разрабатывает ваш проект по технологии git worktree. В нескольких окнах VS Code одновременно, как сплочённая команда специалистов, агенты работают и обсуждают между собой рабочие вопросы. Вы — владелец, заказчик — остаётесь визионером. Ваши ИИ-агенты выполняют ваш заказ на ваш проект.*
>
> *— Николай Кривуша о версии KAIF 2.4*

> *Очень масштабная версия, наделяющая KAIF силой умной системы, самостоятельно развивающейся по циклу обратной связи. Уроборос символизирует замыкание цикла и совершенство. Глаз символизирует то, что KAIF теперь под наблюдением проектов, которые им пользуются, а проекты, которые им пользуются, — под наблюдением KAIF. Все циклы — и метафорические, и технические — замыкаются.*
>
> *— Николай Кривуша о версии KAIF 2.2*

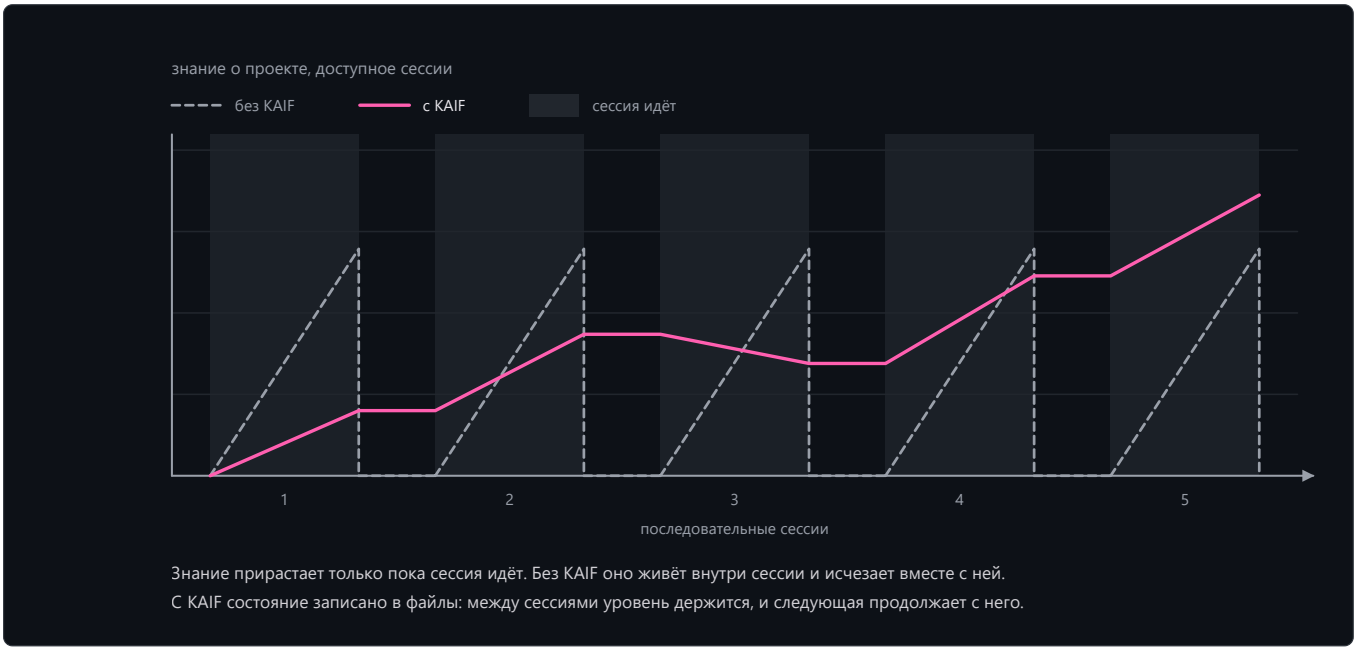
1. Общие сведения

1.1. Основные положения

1. KAIF (Krinik AI Framework) — методологический фреймворк для ИИ-агентов: внешняя память и дисциплина проекта, упакованные в markdown-файлы, которые агент читает и ведёт.
2. Кода во фреймворке нет. Фреймворк — рабочий процесс, записанный файлами: ключевые документы, конвенции директорий и повторяемые навыки. Он подходит любому языку, любому стеку и любой сфере — программированию, науке, дизайну, бизнесу.
3. Индустрия сошлась на формуле для этого слоя: агент = модель + харнесс. Так как модели сравнялись по качеству, большую часть работы теперь делает харнесс вокруг модели. KAIF — внешний харнесс, собранный в markdown: он переносится между агентскими системами.
4. Фреймворк лечит два хронических отказа ИИ-агентов. Потеря контекста: без внешней памяти каждая новая сессия заново открывает архитектуру, решения и недоделанную работу. Дрейф: агент, оставленный без присмотра, либо буксует, либо

берётся за решения, которые принимать не вправе.

5. Человек остаётся визионером, агент исполняет. Решения, которые формируют продукт (бренд, UX, архитектура, нейминг), владелец принимает через документы интервью; всё, что дёшево откатить, агент решает сам.
6. Сырому доверия нет. Работа идёт по fable-циклу, всё созданное несёт маркер тест-статуса (`[NOT-TESTED]` / `[TESTED: …]`), а состязательный судья перепроверяет заявления, прежде чем работу засчитают сделанной.
7. У фреймворка полный жизненный цикл: развёртывание → обновление → форк → удаление. Обновление идёт механически и уважает каждую локальную правку (раздел 6).



1.2. Термины

- **Развёртывание** — проект, в который распакован фреймворк; фиксируется в `.kaif/kaif.json`.
- **Владелец** — человек, которому принадлежит проект; визионер. Владелец заполняет `GOAL.md`, отвечает на интервью и выносит вердикты по вопросам класса «вкус».
- **Агент** — ИИ-система, работающая в проекте (Claude или любая другая); исполнитель.
- **Навык** — повторяемый ритуал, который агент выполняет по команде (`/resume`, `/release`, …); глаголы работы над проектом. Полный набор — Таблица 3.
- **Сфера** — библиотека предметной области (программирование, наука, дизайн, бизнес). Она подгоняет дисциплину фреймворка под терминологию, правила свидетельств и таблицу фродов этого домена.
- **Канон** — обязывающие документы развёртывания (Таблица 1); при расхождении канона и импровизации побеждает канон.

1.3. Границы применения

1. Фреймворк работает на когнитивных проектах, которые ведёт ИИ-агент под человеком-владельцем. Runtime-библиотекой он не служит: внутри целевого продукта ничего не исполняется.
2. Дисциплину держат документы, ритуалы и опциональные машинные стражи. Память модели её не держит: развёртывание, где агент не читает канон перед задачами, пользы не получает.
3. Фреймворк не принимает решений владельца ни при какой широте делегирования: идентичность (имена, слоганы, брендовые строки) и вкусовые вердикты остаются только за человеком.

2. Установка

2.1. Основные положения

1. Вход один — файл `KAIF.md` (15 КБ). Агент читает его и выполняет три шага бутстрапа с принудительными чекпоинтами; файл скачивает установочную машинерию из этого репозитория, и машинерия развёртывает всё механически.
2. Машинерия распаковывает документы, генерирует навыки сразу для пяти агентских систем, локализует документы владельца, подключает указатели автоконтекста, сверяет себя по sha256-манифесту и самоочищается. Единственная

когнитивная работа агента — короткое задание `KAIF_ADAPTATION_TASK.md` : изучить проект, заполнить карты, вывести план.
3. Агент читает `KAIF.md` в 15 КБ. Полное ядро весит 700 КБ, и тонкая установка избавляет агента от его чтения.

2.2. Порядок установки

1. Положите `KAIF.md` в корень проекта.
2. Скажите агенту — своими словами. Годится любая из просьб, и каждая говорит своё:

   > *«Разверни KAIF из KAIF.md» — простая. Поставка на английском, привязка к origin.*

   > *«Разверни KAIF из KAIF.md, мой рабочий язык русский» — документы, которые читаете и заполняете вы, выйдут на вашем языке; сама поставка остаётся английской.*

   > *«Разверни KAIF из KAIF.md, цель проекта я уже написал в GOAL.md» — агент прочитает вашу цель и не будет спрашивать о ней после установки.*

   > *«Разверни KAIF из KAIF.md в анонимном режиме» — без привязки к origin и без упоминаний автора.*

   Агенту понадобятся рабочий Node.js и доступ в сеть к этому репозиторию.
3. Одобрите запуск установщика, если харнесс агента об этом спросит. Он срабатывает на «скачай и исполни» и спрашивает один раз; установщик сверяет каждый скачанный файл с `kaif-manifest.json` по sha256 ещё до запуска.
4. На этом ваша часть кончается — дальше KAIF ставит себя сам. По заданию установки агент подключает напоминания (хуки свежего контекста, раздел 3.4). Сводка установки называет агенту файлы KAIF, которые прячет `.gitignore` вашего проекта (первый коммит оставил бы их за бортом), и для каждого — правило, которое его прячет, и лекарство. Если папка проекта без git, сводка велит агенту выполнить `git init` и закоммитить ваши собственные файлы как есть до коммита файлов KAIF.
5. После установки агент спросит об одном — о цели вашего проекта, если `GOAL.md` не был написан вами заранее, до установки KAIF. `GOAL.md` нужен, чтобы ваш ИИ-агент понимал, над каким проектом идёт работа и чего вы хотите достичь в итоге. Этот вопрос — о том, как адаптировать KAIF под ваш проект.

Установка по умолчанию привязана к origin — именно этим работают проверка версии, уважительные обновления и петля полевых отчётов. Нужно развернуть без привязки к origin — скажите агенту на шаге 2: *«Разверни KAIF из KAIF.md в анонимном режиме»*. Такое развёртывание не несёт ни трекинга origin, ни упоминаний автора (привязанные к origin навыки пропускаются, записка автора вырезается механически, финальный греп-гейт не даёт завершиться, пока остаётся утечка идентичности) и отказывается обновляться по сети. Обновление никогда не переводит развёртывание из режима в режим.

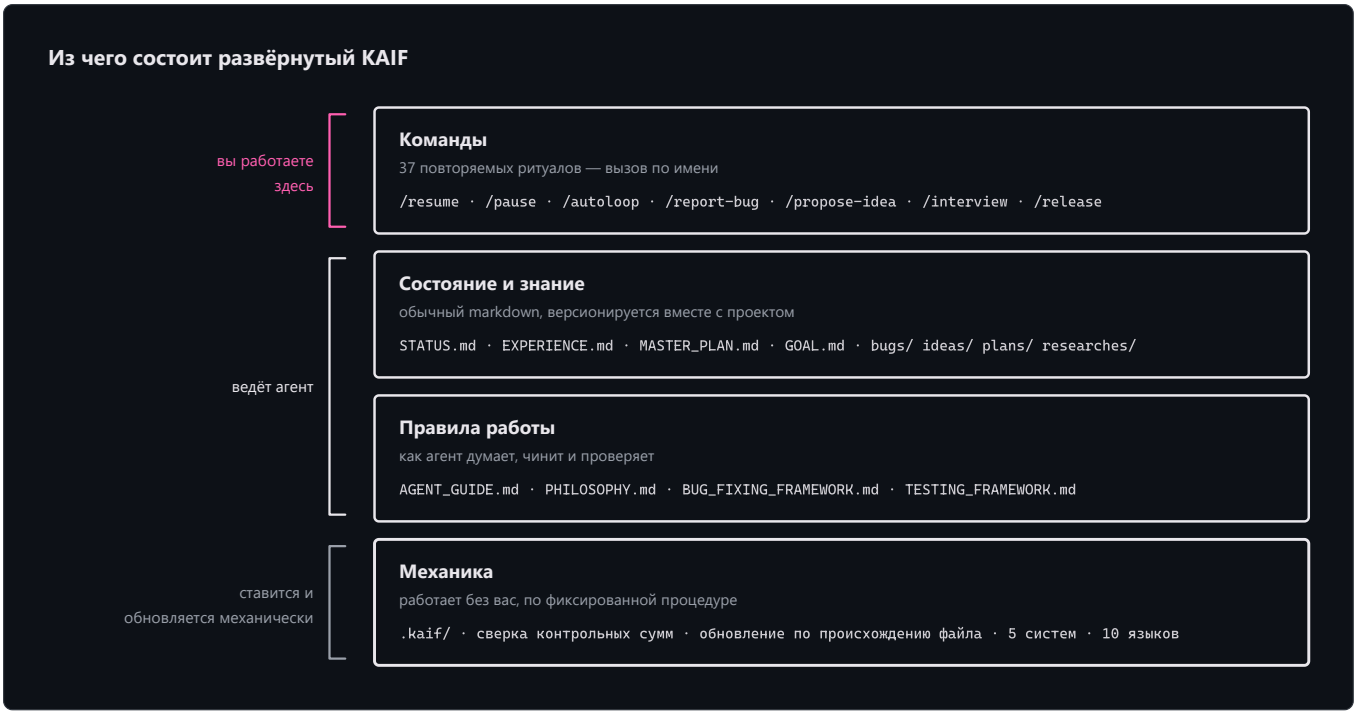
2.3. Офлайн-установка

К каждому релизу прикладывается **`KAIF-FULL.md`** — классическое самодостаточное ядро. Оно распаковывается без сети; это объявленное подмножество полной поставки — языковые пакеты, библиотеки сфер и справочные шаблоны навыков приезжают только сетевым путём. При наличии сети предпочтителен тонкий путь.

3. Устройство развёрнутого фреймворка

3.1. Основные положения

Развёртывание состоит из четырёх слоёв: команды (навыки, вызываемые человеком по имени), состояние и знание (живые документы, которые ведёт агент), правила работы (канон) и механика (`.kaif/` — контрольные суммы, обновление по происхождению, сферы). Любой агентный харнесс несёт пять функций: инструкции, состояние и память, верификацию, гвардрейлы, управление контекстом. Слои KAIF записаны файлами — харнесс переезжает вместе с репозиторием. Слои показаны на схеме ниже.



3.2. Ключевые документы

В корень проекта разворачиваются четырнадцать ключевых документов. Пятнадцатый, портрет голоса владельца, опционален и появляется там, где портрет снят.

Таблица 1 — Ключевые документы развёртывания

Документ	Для чего	Кто пишет / ведёт
`AGENT_GUIDE.md`	Канон: правила, роутер, чек-лист, конвенции	Разворачивает машинерия; агент адаптирует; владелец почти не трогает
`PHILOSOPHY.md`	Как агент мыслит: KISS + Оккам + набор принципов	Универсален — разворачивается дословно
`BUG_FIXING_FRAMEWORK.md`	Как агент отлаживает: гейт намерения, fix→build→test, поиск близнецов	Универсален — разворачивается дословно
`TESTING_FRAMEWORK.md`	Как агент тестирует всё созданное: 7 принципов + контракт `[NOT-TESTED]`/`[TESTED]`	Универсален — разворачивается дословно
`REQUIREMENTS_FRAMEWORK.md`	Как агент пишет и проверяет требования: вектор цели + критерии приёмки первыми, десять критериев качества, стоп-словарь	Универсален — разворачивается дословно
`GOAL.md`	Видение: чего владелец хочет в итоге	**Владелец** — единственный документ, который заполняет он
`STATUS.md`	Живая сводка текущего положения (мягкий ориентир ~200 строк)	Агент, после каждой значимой задачи
`PROJECT_HISTORY.md`	Дописываемая летопись закрытых сессий, фаз, релизов	Агент переносит записи на `/end-chat-soft`

EXPERIENCE.md	Греп-дружелюбный журнал оплаченных уроков	Агент растит сам (/experience)
MASTER_PLAN.md	Фазовый план от текущего состояния к GOAL.md	Агент выводит (/revision)
PROJECT_STRUCTURE_EXTERNAL_MAP.md	Внешняя карта: директории, файлы, связи	Ведёт агент
PROJECT_ARCHITECTURE_INTERNAL_MAP.md	Внутренняя карта: абстракции и их взаимодействия	Ведёт агент
KAIF_FRAMEWORK.md	Запись о развёртывании: какой KAIF развёрнут в проекте и как	Агент пишет после инъекции
KAIF_REFERENCE.md (в .kaif/)	Полная пояснительная записка фреймворка; /help-kaif цитирует её разделы	Разворачивается дословно
AUTHOR_STYLOMETRY.md — **опциональный**	Портрет письменного голоса владельца: регистры, правила, анти-портрет ИИ-маркеров, чек-лист самопроверки. Текст, который владелец подписывает, держится по нему	Снимает /owner-voice с собственных текстов владельца; принимает владелец.

3.3. Директории знаний

Разворачиваются семь директорий знаний, каждая со своим локализованным README. Закрытые единицы в `bugs/`, `ideas/`, `plans/` и `homeworks/` получают тег `DONE` в имени файла; живые справочники тегом не помечаются.

Таблица 2 — Директории знаний

Директория	Что в ней накапливается
plans/	Операционные планы агента и лестницы эпиков
ideas/	Предложения фич — в основном владельца; реализуются только после одобрения
bugs/	По документу на дефект, с форензикой
researches/	Разведдоки и ответы на большие трудные вопросы
interviews/	Решения уровня владельца; владелец отвечает прямо в документе
homeworks/	Задачи, которые может выполнить только человек с телом, включая артефакты класса «вкус»
reports/	Отчёты агента о когнитивно ёмкой работе; обязательные полевые отчёты об обновлении/установке KAIF (KAIF_UPDATES/) и аудит-отчёты сильных моделей (KAIF_AUDIT/)

3.4. Механика

1. В `.kaif/` находятся маркер развёртывания (`kaif.json`), машинерия (`kaif-core.mjs`, на которой держатся ручки `npm run kaif:*`), библиотеки сфер и поставляемые шаблоны (например, скелет портрета голоса владельца `_owner-voice-template.md`).
2. Машинерия разбирает каждый развёрнутый файл по происхождению: template-sha помнит, каким был шаблон, disk-sha — каков файл сейчас. По этому разбору обновление (раздел 6) и сливает файлы, модуль за модулем.
3. Обновление завершается проверкой `update-verify`: посистемные копии навыков ресинкаются с канонических `.claude/skills/`, плейсхолдеры пересканируются, маркер развёртывания самовосстанавливается, и рядом остаётся расписка `.kaif/last-update.json`.
4. В `.kaif/hooks/` лежит модуль **refresh-hooks** для харнессов с lifecycle-хуками: приказ перечитать канон после сжатия контекста, таймер возраста маркера освежения, мягкий страж `STATUS.md` раз в сессию и приказ исполнить `/resume`, когда сообщение владельца открывается словом `resume`. Задания установки и обновления подключают модуль командой `wire-hooks`; владелец отказывается от хуков ключом `"hooks": "off"` в `.kaif/kaif.json`, и развёртывание без хуков не краснеет ни на одном гейте. Хук `pretool-owner-word.mjs` из версии 2.8 останавливал

работу агента на каждом сообщении владельца посреди хода; в версии 2.9 он удалён из поставки: обновление удаляет его нетронутую копию и убирает его запись из `.claude/settings.json`.

4. Навыки

4.1. Основные положения

1. Навык вызывается по имени (`/resume`) или естественной фразой на языке владельца («сделай релиз», «haz un release», …). Алиасы-триггеры на десяти языках дописываются при развёртывании; тела навыков остаются английскими.
2. Навыки генерируются сразу для пяти агентских систем — Claude Code, Codex, Grok Build, Cline, Zoo Code — плюс универсальный `AGENTS.md`; канонические копии живут в `.claude/skills/`.
3. Разворачиваются тридцать семь навыков.

Таблица 3 — Навыки

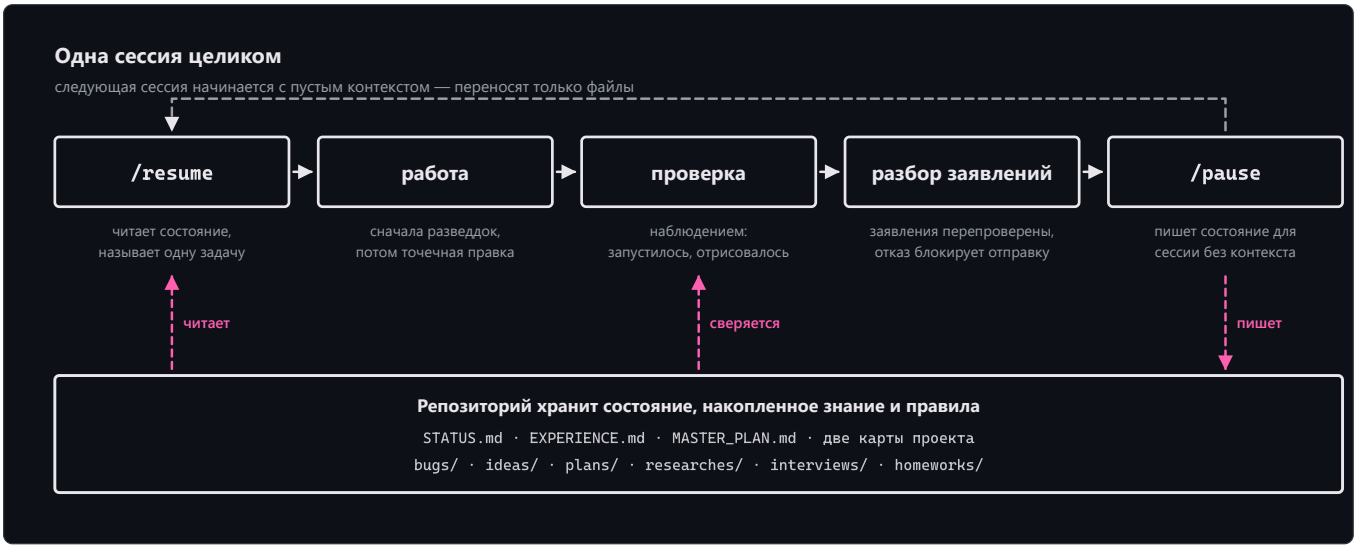
Навык	Назначение
`/resume`	Начать сессию: прочитать канон-документы, выбрать одно главное и объявить его. Объём работы задаёт сообщение владельца под словом `resume`: на вопрос — ответ без работы сверх него, на задачу — задача, на голое `resume` — выбранное главное.
`/kaif-go`	Слеш-команда «продолжай» (короткий алиас `/go`): продолжить работу в текущем чате. Никогда не бланковое «да» на развилки видения, write-gate и строки `AUTH:`.
`/pause`	Мягко припарковать чат: дойти до логической точки, оставить дерево зелёным, продолжить позже в текущем чате — без пушей и церемоний.
`/end-chat-soft`	Мягко закрыть чат: доделать текущую работу до логического среза, затем неторопливо обновить `STATUS.md`, пересобрать артефакты, закоммитить И запушить, передать эстафету другим чатам.
`/end-chat-force`	Срочно закрыть чат прямо сейчас: зафиксировать только то, что нельзя потерять (статус + эстафета), закоммитить И запушить; пропущенные церемонии становятся явной строкой долга.
`/autoloop`	Длинная автономная серия по беклогу; каждый пункт завершается обязательным judge-проходом.
`/dayloop`	Дневная автономная работа, пока владелец занят; в чат идут короткие сводки.
`/nightloop`	Автономная работа до утра; утренний отчёт — результатом вперёд.
`/guarded-loop`	Автономный цикл под сторожем: внешние пробуждения каждые N минут, heartbeat-файл реального прогресса, политика рестартов с потолком эскалации.
`/refresh-context`	Перечитать мастер-план, карты и открытый беклог посреди марафона — восстановить картину.
`/check-backlog`	Ревизия `bugs/` + `ideas/` + `plans/`: что открыто, сделанному — тег `DONE`.
`/experience`	Зафиксировать урок в `EXPERIENCE.md` — или вспомнить уроки по тегам перед задачей.
`/report-bug`	Завести документ дефекта в `bugs/` по канону — один файл на баг.
`/bug-research`	Глубокое исследование без правок кода — обязательно после 3 неудачных слепых фиксов.
`/propose-idea`	Предложить фичу документом в `ideas/` — реализация только после одобрения владельца.
`/interview`	Задать владельцу судьбоносные вопросы A/B/C/D — решения видения не угадываются.
`/owner-voice`	Стилометрический портрет письменного голоса владельца, снятый с собственных текстов владельца в опциональный `AUTHOR_STYLOMETRY.md`; дальше ИИ-текст в артефактах владельца пишется или перепевается так,

	чтобы звучать как владелец, и его инварианты проверяет машина.
`/owner-reviews`	Опциональный контур согласований: интервью и исходящие черновики рендерятся локальными HTML-страницами, решения фиксируются с автором и временем, отправки механически загейчены одобрением — fail-closed.
`/team-deployment`	Спроектировать и развернуть КОМАНДУ ИИ-агентов: проанализировать профиль работы, предложить обоснованный состав на одобрение владельцу, затем материализовать его изолированными worktree под сгенерированной Конституцией команды и общей статус-доской. В комплекте — контракты ролей, два архетипа и контракт инструмента доски; инструменты строит агент проекта.
`/fix-vision`	Зафиксировать визионерские сообщения владельца из чата в документы, пока не испарились.
`/what-next`	Ранжировать следующие шаги по ценности к видению, когда владелец спрашивает «что дальше?».
`/plan-task`	Спланировать обычную задачу/баг/идею в ОДИН операционный план; тяжёлое передаётся `/plan-epic`.
`/plan-epic`	Спланировать тяжёлый эпик полной лестницей: разведка индустрии + локальная разведка → research-документ → мета-план с фазами → операционный план только ближайшей фазы.
`/revision`	Перевывести `MASTER_PLAN.md` из `GOAL.md` и текущего состояния.
`/derive-styleguide`	Вывести стайлгайд владельца из ЕГО ЖЕ образца — утверждён однажды, дальше его стерегут машинные правила.
`/code-revision`	Периодическая читающая ревизия кодовой базы сильнейшей моделью: параллельные ревьюеры, вооружённые оплаченными классами провалов самого проекта; каждой находке — дословная цитата, и каждая переживает адверсарного скептика — или умирает. Прогон оставляет аудит-отчёты в `reports/KAIF_AUDIT/`, сгруппированные по семействам находок, где каждая находка — контракт, исполнимый более слабой моделью.
`/fable-method`	Цикл исполнения: классифицируй → «готово» → свидетельства → действуй → проверь → доложи. *(вендорено из fable-method, MIT)*
`/fable-loop`	Оркестрованный прогон: параллельные свидетельства, хирургическое исполнение, адверсарные верификаторы.
`/fable-judge`	Адверсарная проверка любого «готово»: VERIFIED / CAVEATS / REFUTED.
`/fable-domain`	Сгенерировать доверенный доменный workflow-бандл (адаптер + ловушка + smoke-eval).
`/help-kaif`	Рассказать владельцу про KAIF в чате — структурный мануал.
`/release`	Выпустить релиз (с подтверждением владельца и обязательным judge-проходом; никогда автономно).
`/kaif-version`	Доложить версию развёрнутого KAIF и проверить origin на новый релиз.
`/kaif-update`	Механическое уважительное обновление из origin — снапшоты содержимого берегут локальные кастомизации.
`/kaif-fork`	Слепок эволюционировавшего KAIF в собственный репозиторий владельца — своя линия.
`/kaif-switch-origin`	Переключить трекинг с форка обратно на официальный origin.
`/kaif-remove`	Уважительное удаление — спрашивается: частично (артефакты знаний остаются) или полностью.

5. Работа над проектом

5.1. Цикл сессии

Сессия начинается с `/resume` (агент читает канон и выбирает одно главное), проходит через работу с верификацией и завершается `/pause` (мягкая парковка) или `/end-chat-soft` (полное закрытие с эстафетой). Состояние переносится файлами, не чатом: следующая сессия начинается с пустого контекста и продуктивна сразу.



5.2. Роли и интервью

1. Владелец — визионер: заполняет `GOAL.md`, кладёт идеи в `ideas/`, отвечает на интервью, выносит вкусовые вердикты. Агент — исполнитель: за ним всё остальное.
2. Всё, чего агент хочет ОТ владельца (развилка, вычитка, одобрение, ответ), живёт только в `interviews/`; вопросы закрытые A/B/C/D с рекомендацией первой, и владелец отвечает прямо в документе. Ответ на отрендеренной странице, в документе или в чате обладает равной силой и фиксируется с автором и временем.
3. Письменный голос владельца воспроизводим: `/owner-voice` снимает стилометрический портрет с собственных текстов владельца в `AUTHOR_STYLOMETRY.md` (опциональный канон-файл в корне), и ИИ-текст в артефактах владельца дальше держится по нему.
4. Вопрос владельцу короткий: одна развилка — один вопрос, предмет решения приведён внутри вопроса. Страница вопросов ведёт себя так, как ждёт владелец: повторное нажатие на выбранный вариант отменяет выбор, абзац документа, разбитый переносами строк, показывается одним абзацем, в ответе владельца работает перенос строки.

5.3. Автономные циклы

Если владелец отсутствует, агент перемалывает беклог автономно: `/dayloop` (со сводками), `/nightloop` (утренний отчёт), `/autoloop` (длинная серия) или `/guarded-loop` (под внешним сторожем с heartbeat-файлом — зависший чат не убьёт прогон молча). Каждый пункт завершается обязательным judge-проходом; заметка владельца на бегу уходит в беклог, а не переключает задачу.

5.4. Дисциплина исполнения

Любую нетривиальную задачу агент ведёт по fable-циклу: классифицируй запрос → определи «готово» → собери свидетельства → реши → действуй хирургически → проверь наблюдением → доложи результатом-вперёд, с принудительными артефактами на точках решения. Когда работа объявлена завершённой (своя или чужая), `/fable-judge` состязательно перепроверяет заявления, прежде чем работу засчитают сделанной. С git агент работает по правилу «одна команда — одно действие»: `git add`, коммит и пуш идут отдельными вызовами: цепочку команд, склеенную через `&&`, классификатор разрешений авторежима отклоняет.

5.5. Гвардрейлы и провенанс

1. Наблюдение побеждает домысел: у каждого заявления назван источник, а пробел в каноне открывает ровно три двери — найти ответ в существующем источнике истины, спросить владельца, выдумать правдоподобное. Третья дверь закрыта.

2. ИИ-текст в канон-артефактах владельца несёт пометки провенанса `[AI]…[/AI]`; пометку снимает только слово владельца. Опциональные машинные инструменты (гейт провенанса, линтер канона) поставляются модулями и подключаются по запросу.
3. Страж текстового правила работает с шумом ~10 ложных срабатываний на 1 настоящее; исключения явные, с причиной в строке. Ложный маркер `[TESTED]` судья считает фродом.

5.6. Свежий контекст

1. Агент перечитывает ядро канона по четырём триггерам: час живой работы, старт тяжёлой задачи, сжатие контекста или длинный простой, точки ритуалов (`/resume` , `/refresh-context` , каждая итерация циклов). Перечитывание нужно, так как правила, прочитанные один раз на старте сессии, тают по мере заполнения и сжатия контекста: длинная сессия держит в голове пересказ канона.
2. Освежение — проверяемое действие. Свидетельство двухчастное, и обе части обязательны: маркер `.kaif/refresh-marker.json` (момент, что перечитано, какой триггер) и цитата-приёмка в чате — одна конкретная строка из перечитанного, относящаяся к задаче в работе. Маркер без цитаты — фрод того же класса, что ложный `[TESTED]` .
3. Агент знает свою машину из собственных проб: досье окружения в `AGENT_GUIDE.md` фиксирует, чем являются шеллы, тулчейн и кодировки этой машины, держит рядом с каждым фактом команду пробы и объявляет факты старше четырёх недель вновь гипотезами.
4. Хуки свежего контекста находят корень проекта и тогда, когда агент ушёл в подпапку: напоминание хука относится к проекту, в котором работает агент.

6. Обновление, форк, удаление

6.1. Основные положения

1. Обновление выполняется механически и уважительно: `npm run kaif:update` (или `node .kaif/kaif-core.mjs update`) получает свежую машинерию и классифицирует каждый файл фреймворка по происхождению. Нетронутый локально файл обновляется целиком; адаптированный или локализованный проходит помодульное слияние — локальные модули остаются локальными, не тронутые локально модули апстрима молча получают новый текст шаблона, и модуль попадает в короткое задание `KAIF_UPDATE_TASK.md` (с готовым диффом) только там, где апстрим действительно изменился под локальными правками.
2. Содержимое владельца — `GOAL.md` , `STATUS.md` , директории знаний — в объём обновления не входит никогда.
3. Обновление завершается теми же гарантиями, что и свежая установка (раздел 3.4), и оставляет расписку `.kaif/last-update.json` .
4. Задание обновления `KAIF_UPDATE_TASK.md` перечисляет места проекта, которые ещё ссылаются на удалённые или перенесённые навыки и хуки KAIF, и обновление приводит к тексту выпуска указатели контекста других агентских систем: `AGENTS.md` , `CLAUDE.md` , правила Cline и Zoo Code. При обновлении через несколько версий собственные правки проекта помечаются строкой «with local edits», и `update-verify` называет их правками проекта.
5. Правила новой версии принимаются вместе с версией: задание обновления не выносит их владельцу вопросами и называет их в полевом отчёте.

6.2. Ручки

Механические ручки, которые ставятся в `package.json` , собраны в Таблице 4. Форк, переключение origin и удаление делают свои навыки — попросите их у агента.

Таблица 4 — Ручки `npm run kaif:*`

Команда	Что делает
`npm run kaif:version`	Показать версию развёрнутого KAIF (из `.kaif/kaif.json`).
`npm run kaif:check`	Сверить развёртывание с манифестом — работает и после самоочистки.
`npm run kaif:update`	Механическое уважительное обновление из origin (раздел 6.1).

6.3. Обновление старых развёртываний

Положите свежий тонкий `KAIF.md` поверх развёртывания старше 1.5 и попросите агента обновиться. Установщик найдёт существующее развёртывание и примет всё найденное как локальное; содержимое владельца переживает такое обновление байт в байт.

7. Сферы, агентские системы, языки

7.1. Сферы

Сфера разворачивается в `.kaif/spheres/` с терминологией домена и его дисциплиной исполнения: обязательный минимум свидетельств, порядок авторитетов, что значит «проверено наблюдением», таблица фродов, по которой охотится судья, и ремесленные рецепты домена. Готовые сферы: программирование · наука · дизайн · бизнес; новая сфера пишется при развёртывании по поставляемому шаблону.

7.2. Агентские системы

Навыки генерируются сразу для пяти систем: Claude Code (`.claude/skills/` , канонические) · Codex (`.agents/skills/`) · Grok Build (`.grok/skills/`) · Cline (`.cline/skills/`) · Zoo Code (`.roo/commands/`) — плюс универсальный `AGENTS.md` ; Cursor, Copilot и Windsurf работают через него. Модуль refresh-hooks несёт образцы хук-конфигов только для тех харнессов, чей хук-контракт снят с живой вендорской документации: Codex, Cursor, Copilot, Antigravity; Grok Build совпадает с контрактом Claude Code и своего образца не требует. Там, где у вендора хук-механизма нет, модуль говорит это одной строкой вместо угаданного пути.

7.3. Языки

Исходники фреймворка английские. При развёртывании машинерия локализует документы владельца (`GOAL.md` , `KAIF_FRAMEWORK.md` , README директорий) из готовых пакетов — десять языков: en, ru, zh-Hans, es, hi, ar, pt, fr, de, ja — и дописывает каждому навыку алиасы-триггеры на языке владельца. Внутренние документы агента остаются английскими by design. Остальные языки деградируют честно: английский плюс пункт перевода в задании адаптации.

С версии 2.3 ведутся два пакета: en и ru. Остальные восемь заморожены в полном состоянии 2.2 — они остаются в поставке, разворачиваются как прежде и запинены побайтно стражем, но обновлений не получают. Причина практическая: девять параллельных пакетов тяжелы в поддержке и пока не окупаются. Замороженный пакет оживает по запросу сообщества — откройте issue.

8. Справочные сведения

8.1. Вехи

История в одной таблице; каждое кодовое имя — дисциплина, которую фреймворк выучил. Полные ноты (с 1.1) живут в релизах.

Таблица 5 — Версии

Версия	Кодовое имя	Дата	Выученная дисциплина
v1.0	—	30.06.2026	Дистилляция: рабочий метод извлечён в одно самораскрывающееся ядро; репозиторий обёрнут собственным фреймворком с первого дня.
v1.1	Structured KAIF	01.07.2026	Версионирование `x.y`, набор ключевых документов (видение, план, две карты), директории знаний.
v1.2	Anonymous KAIF	03.07.2026	Механический распаковщик, анонимный режим установки, трансляция навыков для не-Claude систем.
v1.3	Slim KAIF	06.07.2026	Однофайловый лёгкий вариант (снят в 1.5 в пользу тонкого ядра + офлайн `KAIF-FULL.md`).
v1.4	Savvied KAIF	08.07.2026	`EXPERIENCE.md` — греп-дружелюбный журнал уроков; ленивая загрузка контекста; опциональные хуки.

v1.5	Tested KAIF	17.07.2026	Тонкая установка, пять агентских систем и десять языков сразу, механические уважительные обновления, `TESTING_FRAMEWORK.md`, вендоренный fable-цикл; полевая сертификация на локальной модели 12B.
v1.6	Homeostatic KAIF	24.07.2026	Гвардрейлы для слабых моделей: наблюдение вместо домысла, три двери, судья перед каждым пушем, пометки провенанса `[AI]…[/AI]`.
v2.0	Excellent KAIF	28.07.2026	Обновление машинерией, а не разумом: карта модулей, template-vs-disk sha, расписки обновления, записка `KAIF_REFERENCE.md`, постоянный песочный полигон.
v2.1	Strong KAIF	31.07.2026	Контур владельца: правило места вопросов с `/owner-reviews`, портрет голоса `/owner-voice`, ремесленные протезы для слабых сессий (`/code-revision`, craft-слоты, `/guarded-loop`), лестница планирования, летопись `PROJECT_HISTORY.md`.
v2.2	Yolden KAIF	08.08.2026	Цикл замыкается: интерактивный контур делает вопрос к владельцу рабочим каналом; у пути сигнала «поле → исток» появились пять предписанных шагов; `REQUIREMENTS_FRAMEWORK.md` входит 14-м ключевым документом; перечитывание канона становится проверяемым действием с маркером-свидетельством и опциональным модулем `refresh-hooks`; `/kaif-go` — это слеш-команда «продолжай»: простой способ продолжить работу в текущем чате.
v2.3	Subjected KAIF	21.08.2026	Версия, которая весь свой цикл была под наблюдением: тринадцать issues от агентов живых проектов стали её скоупом, то есть она сделана из того, что нашли пользователи. Канон заговорил командами — обязательство несёт команду, нумерованный шаг или чекбокс, а `TESTING_FRAMEWORK.md` пересобран вокруг цепочки тестовых активностей; записанный урок обязан ответить, нельзя ли грабли механизировать вместо того, чтобы их помнить; `update` пишет журнал до первой мутации, а убитый прогон дочиняет `resume`; отказ сети на Windows сообщает об одной ошибке вместо двух; восемь языковых пакетов заморожены с объявленными версией, состоянием и причиной, а `en` и `ru` продолжают вестись.
v2.4	Teamed Up KAIF	28.08.2026	Харнесс берёт команду: опциональный навык `/team-deployment` проектирует и разворачивает команду ИИ-агентов — роли, изолированные рабочие места в worktree, сгенерированная Конституция команды, общая доска статусов. Закрытие чата раздвоено на `/end-chat-soft` и `/end-chat-force`, а названное время окончания начинает мягкое закрытие, вместо того чтобы обрывать работу раньше срока. Перед работой агент произносит символ веры и молитву принципов. Интерактивный контур зовёт именованным нейроголосом и рендерит развилки радиокнопками. Вместе с версией едут четыре полевых фикса из отчётов 2.3.
v2.5	Experienced KAIF	04.09.2026	Правила переезжают в точку решения: исключение для дефектов KAIF стоит внутри гейта авторизации, инженерную развилку закрывает четвёртая дверь и принудительная строка `FORK:`, каждый страж объявляет угрозу, против которой доказан, защищённый цикл записывает свою границу. Доставка считается одной метрикой владельца в каждом закрытии сессии, у дефектов есть лестница тяжести, у перечитываемого ядра — бюджеты строк. Тикет о KAIF доставляет команда машинерии. Командный навык принимает живую команду, даёт доске четыре состояния и везёт CI вместе с командой. Обновление держит якорный блок целым, печатает вердикты с числами и связывает себя репетицией, пишет пункт `stale-claims` безусловно и сканирует скрипты проекта. Критерий приёмки можно записать на языке владельца четырёхстрочным сценарием — Ситуация · Действие · Результат · Проверка — с опциональным линтером его формы.
v2.6	Mindful KAIF	06.09.2026	Агент работает по меркам владельца: вопрос владельцу — приоритет номер один, а его показ — записанный факт; каждый вопрос и вариант — сценарий того, что владелец увидит, на языке заказчика; вектор доставки выводится из реестра систем и не спрашивается; репетиция обновления дошла до bootstrap-маршрута; интерактивный контур входит в поставку — контракт и генератор трёх лиц; свежая

			реплика записывается, но не ранжируется; «готово» о продакшене — после реального мира владельца; поставка не называет проекты-доноры.
v2.7	Audited KAIF	18.09.2026	Записи агента становятся проверяемыми: слово владельца цитируется, слово агента подписывается; вопрос владельцу задаётся после поиска по его прежним ответам; «тест» — это функциональный прогон, и каждый прогон оставляет отчёт; текст владельца пишется по портрету его голоса; страница владельца сохраняет его ответ; ложь исправляется там, где оставлена; урок, повторившийся дважды, краснеет; бюджет документов, которые агент перечитывает в начале сессии, считается по строкам, которые написал сам проект.
v2.8	Noble KAIF	26.09.2026	KAIF становится легче и слышит владельца: агент отвечает на слово владельца, пока работает, и работает дальше; отвеченные решения владельца идут раньше плана; страница вопросов владельца записывает ответы по одному; обновление ничего не теряет молча; сканеры видят проект так, как его видит git; канон держит каждое правило, истории правил переехали в пояснительную записку `KAIF_REFERENCE.md`; баг-репорт тестировщика получил обязательную форму, и «не воспроизвелось» пишется только после охоты за шагами.
v2.9	Distilled KAIF	16.10.2026	Поставка новой версии подключается и принимается сразу: задания установки и обновления сами подключают напоминания, правила новой версии принимаются без интервью; хук, который останавливал агента на сообщении владельца посреди хода, удалён. Обновление называет места проекта, которые ссылаются на удалённые навыки и хуки, обновляет указатели других агентских систем и при обновлении через несколько версий помечает собственные правки проекта. Страница вопросов ведёт себя так, как ждёт владелец; вопрос владельцу короткий; на вопрос под словом `resume` агент отвечает и работу сверх ответа не начинает; линтер авторства не уносит личные файлы владельца в общий коммит; для git одна команда делает одно действие.

8.2. Структура репозитория

```
KAIF.md                                  ⭐ тонкая точка входа (бутстрап + встроенный загрузчик), генерируется
KAIF_REFERENCE.md                        полная пояснительная записка (генерируется из framework/KAIF_REFERENCE.md)
README.md                                это руководство (EN+RU)
README.pdf                               его отрендеренная копия
LICENSE                                  MIT
KAIF.jpg                                 логотип
framework/                               канонические универсальные шаблоны (полезная нагрузка)
  _intro.md                              нарратив полного ядра
  installer/                             KAIF-CORE.mjs (машинерия) · KAIF-LOADER.mjs · нарратив тонкого ядра
  skills/                                37 шаблонов навыков (по директории на навык)
  spheres/                               библиотеки сфер: programming · science · design · business · _template · _index
  adapters/                              10 адаптеров агентских систем (пять целевых для навыков + фолбэки и архивные)
  templates/_owner-voice-template.md     скелет портрета голоса владельца (едет в .kaif/)
  templates/languages/                   9 языковых пакетов (документы владельца + алиасы навыков; английский — исходник)
  tools/                                 10 опциональных tool-модулей: гейт провенанса и девять линтеров (канон, требования,
                                         стражи, сценарии, ранжирование, авторство, отчёты прогона, голос, опыт) + генератор контура contour/
  hooks/                                 модуль refresh-hooks (4 скрипта + образцы конфигов → .kaif/hooks/)
  readmes/                               7 README директорий
  AGENT_GUIDE.md … KAIF_REFERENCE.md     шаблоны четырнадцати ключевых документов
dist/                                    генерируемая поставка (руками не правится)
```

```
  KAIF.md                          тонкая точка входа
  KAIF-CORE.mjs                    установочная машинерия
  KAIF-CORE-BUNDLE.md              бандл полезной нагрузки (файловые блоки)
  kaif-manifest.json               sha256-манифест
  KAIF-FULL.md                     офлайн самодостаточное ядро
  kaif-module-map.json             карта модулей (заголовки → модули)
assets/                            генерируемые схемы README (3 × light/dark × EN/RU)
tools/                             build-framework.mjs · check-framework.mjs · sandbox-suite.mjs (полигон)
                                   · module-map-lib.mjs · build-diagrams.mjs · readme-pdf.mjs · commit.mjs ·
kaif.mjs
AGENT_GUIDE.md · STATUS.md · …     обвязка для dogfooding: фреймворк, применённый к самому себе
plans/ ideas/ bugs/ researches/    директории знаний обвязки
interviews/ homeworks/ reports/    (в каждой свой README)
```

8.3. Этот репозиторий фрактален

Этот репозиторий и есть фреймворк, и он же обёрнут фреймворком — он применяет себя к себе. В корне живут рабочие `AGENT_GUIDE.md`, `STATUS.md`, `.claude/skills/` и директории знаний, по которым идёт разработка самого фреймворка. Развёртывание в другой проект начинается только с `KAIF.md` — не с файлов обвязки этого репозитория. Всё нужное развёртыванию машинерия берёт из `dist/`, который генерируется из `framework/` командой `node tools/build-framework.mjs`; сгенерированные артефакты руками не правятся.

8.4. Ограничения текущей версии

1. Локализация покрывает документы владельца и алиасы-триггеры навыков (десять языков); канон и тела навыков — английские by design.
2. Родные навыки генерируются для пяти агентских систем; остальные харнессы (Cursor, Copilot, Windsurf) работают через универсальный `AGENTS.md` без родных файлов навыков.
3. Песочный полигон (28 сводов) проверяет машинерию развёртывания и обновления; сама методология проверяется полевыми отчётами, не полигоном.
4. Дисциплина держится на документах и ритуалах; без опциональных tool-модулей и хуков runtime-принуждения нет — агент, пропустивший `/resume`, работает без канона.
5. В поставке 14 документов + 7 README + 37 навыков + 1 распаковщик = 59 встроенных файлов; 184 блока бандла; 836 модулей.

Лицензия

MIT — © 2026 **Mikalai Kryvusha** aka **KOT KRINIK** · Николай Кривуша aka Кот Криник. Навыки дисциплины исполнения (`fable-*`) вендорены из fable-method © Sahir619, MIT.

Используйте, копируйте, меняйте, поставляйте — в том числе на проекте самого фреймворка. Спасибо и приятной работы!